

ZroAct AI 서버 연동 구조 보고서

1. 문서 목적

이 문서는 백엔드 서버와 AI 서버를 어떻게 연결할지 설명하기 위한 구조 정리 문서이다. 사용자는 CCTV 영상 또는 저장된 영상을 백엔드에 전달하고, 백엔드는 해당 영상을 AI 서버에 업로드한다. AI 서버는 Stage 1 과 Stage 2 를 순서대로 실행한 뒤, VLM 이 판단한 request 별 위험 로그를 백엔드가 조회할 수 있는 형태로 제공한다.

현재 구현은 실제 실시간 서버의 최종 형태라기보다는, 백엔드 연동을 테스트하기 위한 MVP 구조이다. 다만 기존 실험 코드와 동일한 모델 흐름을 사용하므로, 이후 실시간 worker 구조로 확장할 때도 같은 데이터 흐름을 유지할 수 있다.

2. 전체 연결 구조

```
사용자 / CCTV 관제 PC
→ 백엔드 서버에 영상 업로드
→ 백엔드 서버가 AI 서버 API 로 영상 전달
→ AI 서버가 job_id 생성
→ AI 서버 내부에서 Stage 1 실행
→ Stage 1 결과를 이용해 Stage 2 request 구성
→ Stage 2 VLM이 normal / unsafe / danger 판단
→ request 별 VLM 로그 저장
→ 백엔드가 /jobs/{job_id}/logs 조회
→ 프론트엔드 CCTV 화면과 이벤트 로그에 표시
```

중요한 점은 사용자가 AI 서버에 직접 접근하는 구조가 아니라는 것이다. 백엔드 서버가 영상 업로드, job 관리, 결과 조회를 담당하고, AI 서버는 모델 추론과 로그 생성만 담당한다.

3. AI 서버 내부 파이프라인

AI 서버는 하나의 영상을 받으면 내부적으로 다음 단계를 순서대로 실행한다.

1. 입력 영상 저장
2. 30FPS 프레임 추출
3. Stage 1 YOW0v3 Action Detection
4. Stage 2 용 3-frame sequence 생성
5. Stage 2 VLM Risk Classification

- 6. request 별 VLM 로그 생성
- 7. 백엔드 조회용 파일 및 API 응답 구성

단계	역할	출력
프레임 추출	입력 영상을 30FPS 기준 프레임 이미지로 변환	frame image 목록
Stage 1	사람 위치와 행동 라벨 생성	bbox, action label
Stage 2 request 구성	VLM 에 넣을 3 장짜리 sequence 구성	[t, t+10, t+20] request
Stage 2	3 장의 이미지와 action 정보를 보고 위험 상태 판단	normal / unsafe / danger
serving 로그 정리	백엔드가 조회할 수 있는 로그 형태로 정리	/jobs/{job_id}/logs 응답

4. Stage 1: YOWOv3 Action Detection

Stage 1 은 CCTV 프레임에서 사람을 찾고, 각 사람에 대해 bounding box 와 action label 을 생성하는 단계이다. 이 단계의 목적은 최종 위험도를 판단하는 것이 아니라, Stage 2 가 상황을 이해할 수 있도록 시각적 근거와 행동 단서를 제공하는 것이다.

Stage 1 입력과 출력

- 입력: 30FPS 로 추출된 프레임 이미지
- 출력: 사람 bounding box, action label
- 예시 action: stand, walk, carry/hold, bend/bow

Stage 1 처리 간격

모든 프레임을 Stage 1 에 넣으면 처리 시간이 커진다. 그래서 현재 통합 구조에서는 sample_rate=10 으로 설정했다. 즉 30FPS 영상 기준으로 10 프레임 간격, 약 0.33 초마다 한 번 Stage 1 결과를 생성한다.

30FPS 입력 기준:
frame 000016
frame 000026
frame 000036
...

Stage 1 처리 간격:

```
sample_rate = 10
```

이렇게 하면 모든 프레임을 처리하는 것보다 속도를 줄일 수 있고, Stage 2 가 사용하는 `[t, t+10, t+20]` 구조와도 자연스럽게 연결된다.

5. Stage 2: VLM Risk Classification

Stage 2 는 Stage 1 결과를 바탕으로 3 장의 이미지를 묶어서 VLM 에 입력한다. VLM 은 이미지 3 장과 각 프레임의 action label 을 함께 보고 현재 sequence 의 위험 상태를 판단한다.

Stage 2 입력 구성

Request 예시:

```
F1: frame=16, time=0.533s, action=stand, carry/hold
F2: frame=26, time=0.867s, action=stand, carry/hold
F3: frame=36, time=1.200s, action=stand, carry/hold
```

VLM 입력:

이미지 3장 + frame/time 정보 + Stage 1 action label + prompt

Stage 2 출력

Stage 2 의 출력은 세 가지 위험 상태 중 하나이다.

- **normal**: 펜스 침입과 관련 없는 일반 행동
- **unsafe**: 펜스 근처 접근, 확인, 접촉, 대기 등 침입 전 의심 상태
- **danger**: 발을 들어 오르려 하거나, 발판/펜스/단 위에 발을 올리거나, 경계를 넘기 시작한 상태

Stage 2 VLM 은 request 하나가 끝날 때마다 `risk_state` 를 만든다. 이 request 별 결과가 백엔드 로그의 기본 단위가 된다.

6. 백엔드가 가져갈 로그 구조

백엔드가 프론트엔드에 로그를 띄우기 위해 필요한 정보는 복잡하지 않다. 각 VLM request 마다 어떤 시간대에서 어떤 위험 상태가 나왔는지, 그리고 근거로 볼 이미지와 action 이 무엇인지 정도면 충분하다.

```
{
  "job_id": "test_001",
```

```

"offset": 0,
"count": 1,
"next_offset": 1,
"total": 12,
"logs": [
  {
    "video_id": "intrusion_climb-over-fence_rgb_0007_cctv3",
    "request_id": "intrusion_0007_f000016_000026_000036",
    "frame_indices": [16, 26, 36],
    "time_sec": 1.2,
    "risk_state": "danger",
    "actions": [
      "stand, carry/hold (an object)",
      "stand, carry/hold (an object)",
      "stand, carry/hold (an object)"
    ],
    "images": ["...", "...", "..."],
    "latency_sec": 1.56
  }
]
}

```

필드	의미
job_id	영상 처리 작업 ID
request_id	Stage 2 VLM request ID
frame_indices	VLM 에 들어간 3 장의 프레임 번호
time_sec	해당 request 가 영상에서 어느 시점인지 나타내는 시간
risk_state	normal, unsafe, danger 중 하나
actions	Stage 1 에서 나온 행동 라벨
images	프론트에서 근거 이미지로 보여줄 수 있는 이미지 경로

result.json 은 전체 처리 결과를 확인하기 위한 최종 요약/디버깅용이다. 실제 프론트엔드 로그 표시의 중심은 /jobs/{job_id}/logs 에서 가져오는 VLM request 별 로그이다.

7. API 사용 방식

서버 실행

```
cd /home/capstone2/zroact-stage2
/home/capstone2/miniconda3/envs/qwen35/bin/python -m uvicorn \
  serving.app:app \
  --host 0.0.0.0 \
  --port 9000
```

영상 업로드

```
curl -X POST http://AI_SERVER_IP:9000/jobs \
  -H "X-API-Key: change-this" \
  -F "file=@/path/to/input.mp4"
```

처리 상태 조회

```
curl http://AI_SERVER_IP:9000/jobs/{job_id} \
  -H "X-API-Key: change-this"
```

VLM request 별 로그 조회

```
curl "http://AI_SERVER_IP:9000/jobs/{job_id}/logs?after=0&limit=100" \
  -H "X-API-Key: change-this"
```

최종 요약 조회

```
curl http://AI_SERVER_IP:9000/jobs/{job_id}/result \
  -H "X-API-Key: change-this"
```

8. 현재 테스트 결과

단일 테스트 영상에 대해 통합 파이프라인을 실행했고, Stage 1 과 Stage 2 가 정상적으로 연결되는 것을 확인했다. 해당 테스트에서는 Stage 2 request 12 개가 생성되었고, request 별 로그를 /jobs/{job_id}/logs 형태로 조회할 수 있다.

항목	결과
입력 영상	intrusion_climb-over-fence_rgb_0007_cctv3.mp4
Stage 1 처리 간격	sample_rate=10, 10 프레임 간격
Stage 2 request 수	12 개
최종 위험 상태	danger
VLM request 평균 시간	약 1.56 초
전체 처리 시간	약 48.2 초

9. 실시간성 개선 방향

현재 MVP 는 영상 하나를 job 단위로 처리하고, 내부에서 pipeline 을 실행한다. 그래서 모델 로딩 시간이 포함되어 완전한 실시간 구조는 아니다. 실시간성을 높이려면 Stage 1 과 Stage 2 모델을 worker 형태로 한 번만 로드해두는 구조가 필요하다.

현재 MVP:
 영상 업로드
 → pipeline 실행
 → Stage 1 / Stage 2 모델 실행
 → request 별 로그 조회

향후 실시간 구조:
 Stage1Worker: Y0W0v3 상시 로드, 10 프레임 간격 처리
 Stage2Worker: VLM 상시 로드
 Scheduler: [t, t+10, t+20] sequence 생성
 Backend: VLM 로그를 실시간 또는 준실시간으로 수신

10. 백엔드 담당자에게 전달할 내용

- 백엔드는 영상을 AI 서버의 POST /jobs 로 업로드한다.
- AI 서버는 Stage 1 을 10 프레임 간격으로 돌려 bbox/action 을 만든다.
- Stage 2 는 3 장의 이미지와 action label 을 이용해 normal / unsafe / danger 를 판단한다.
- 프론트 로그에 쓸 데이터는 GET /jobs/{job_id}/logs 로 가져간다.

- `result.json`은 최종 요약/디버깅용이고, 실제 로그 표시의 중심은 request 별 VLM 로그이다.
- 현재는 백엔드 연동 테스트용 MVP 이며, 추후 worker 구조로 바꾸면 지연 시간을 줄일 수 있다.